

2026학년도 1학기 인공지능 과제 1 안내문

8-Puzzle 탐색 알고리즘 구현 및 실험 분석

본 문서는 학생 배포용 공식 안내문이며, 구현 범위, 실험 기준, 제출 형식 및 보고서 작성 기준을 함께 제시합니다.

1. 과제 개요

과제명	8-Puzzle 탐색 알고리즘 구현 및 실험 분석
제출 파일	search.py, 보고서 PDF
수정 허용 파일	search.py 만 허용 (그 외 파일 수정 금지)
구현 대상	DFS, BFS, UCS, heuristic(), A*
권장 비교 대상	BFS, UCS, A*
필수 수행	Task 1 ~ Task 9 전체

본 과제의 핵심은 단순 구현이 아니라, 실험 결과를 근거로 알고리즘의 특성과 차이를 분석하여 설명하는 데 있습니다.

2. 제공 파일 및 역할

파일명	설명
search.py	학생이 직접 구현하여 제출해야 하는 파일
eightpuzzle.py	8-Puzzle 상태, 이동 규칙, goal test 정의
main.py	콘솔 실행 파일
main_ui.py	브라우저 기반 시각화 실행 파일
01. report_template.docx	보고서 작성용 Word 템플릿

배포된 search.py는 과제용 템플릿입니다. 따라서 알고리즘 구현 이전에는 콘솔 실행 또는 UI 실행이 완전히 동작하지 않을 수 있습니다.

학생은 **search.py** 파일만 수정해야 합니다. eightpuzzle.py, main.py, main_ui.py, 안내문/템플릿 파일 수정은 허용되지 않습니다.

3. 실행 안내

콘솔 실행

```
python3 main.py
```

브라우저 UI 실행

```
python3 main_ui.py
```

브라우저 UI 사용 절차

- New Puzzle 버튼을 눌러 랜덤 퍼즐을 생성합니다.
- 원하는 알고리즘 버튼을 선택합니다.
- 해결 과정이 애니메이션으로 표시됩니다.
- Stop으로 애니메이션을 중지하고, Reset으로 초기 상태로 되돌릴 수 있습니다.

4. 구현 대상 및 실험 유의사항

구현 대상 알고리즘

- DFS**: 가장 깊은 노드부터 탐색하며, 최적 해를 보장하지 않습니다.
- BFS**: 가장 얕은 노드부터 탐색하며, 동일 비용 조건에서 최단 경로를 보장합니다.
- UCS**: 누적 비용이 가장 낮은 노드부터 탐색합니다.
- A***: 경로 비용과 휴리스틱을 함께 고려하여 탐색합니다.

권장 비교 대상

- BFS
- UCS
- A*

주의해서 다룰 알고리즘

- DFS: goal을 찾더라도 긴 경로를 반환할 수 있음
- Random Search: 결과와 실행 시간이 불안정함

구현 시 유의할 점

- Graph search 방식으로 구현해야 합니다.
- 이미 방문한 상태를 불필요하게 반복 탐색하지 않아야 합니다.
- 반환값은 goal state에 도달하기 위한 action 리스트여야 합니다.
- 가능한 action은 up, down, left, right입니다.

5. AI 사용 규정

AI로 가능한 일

- 코드 구조 이해
- 실험 아이디어 정리
- 문장 다듬기
- 표와 그래프 정리 보조

학생이 직접 해야 하는 일

- 코드 실행
- 실험 결과 확인
- AI 설명 검증
- 최종 결론 작성

보고서 마지막에는 반드시 **AI 사용 공개** 항목을 포함하고, 사용한 도구, 사용 목적, 직접 검증한 내용, 수정한 설명 여부를 명시해야 합니다.

6. 제출 안내

제출물

- search.py
- 보고서 PDF

권장 제출 파일명

```
[분반]_[이름]_[학번]_search.py
[분반]_[이름]_[학번]_report.pdf
```

보고서 필수 기재 항목

- 제목
- 이름 및 학번
- 실행 환경
- 실험 설정
- 실험 결과
- 분석
- 결론
- AI 사용 공개

7. 보고서 작성 형식

- 표지 정보**: 과제명, 과목명, 분반, 이름, 학번, 제출일
- 서론**: 8-Puzzle 문제 소개 및 분석 목적
- 실행 환경**: 운영체제, Python 버전, 실행 방식, timeout 여부
- 실험 방법**: 퍼즐 생성 방법, 실험 개수, 측정 방식, 정리 방식
- 실험 결과**: 표 또는 그래프 중심으로 정리
- 분석**: 알고리즘별 차이와 그 이유 설명
- 결론**: 핵심 결과 및 학습 내용 정리
- AI 사용 공개**

8. 필수 수행 과제

Task 1. 핵심 파일 역할 설명

목표: 어떤 파일을 왜 쓰는지 스스로 설명할 수 있어야 합니다.

- search.py, eightpuzzle.py, main.py, main_ui.py를 모두 열어 핵심 함수/클래스를 확인합니다.
- 각 파일을 *역할 1문장 + 과제에서의 사용 시점 1문장*으로 정리합니다.
- 보고서에 search.py는 구현 대상, 나머지 파일은 분석 대상임을 구분해 작성합니다.

보고서 필수 포함: 파일별 역할표(최소 4행), 본인 구현 중심 파일 명시.

Task 2. 알고리즘 동작 검증

목표: DFS/BFS/UCS/A*가 실제로 goal까지 가는지 확인합니다.

- 서로 다른 퍼즐 최소 3개를 준비합니다(쉬움/중간/어려움 권장).
- 각 퍼즐에 대해 DFS/BFS/UCS/A*를 모두 실행합니다.
- 각 실행마다 성공 여부, 경로 길이, 에러 발생 여부를 기록합니다.
- 실패가 있으면 실패 입력, 원인, 수정 후 재실행 결과를 함께 남깁니다.

보고서 필수 포함: 검증 로그 표(퍼즐 ID, 알고리즘, 성공/실패, 비교).

Task 3. 실행 시간 비교

목표: BFS/UCS/A*의 시간 성능을 공정하게 비교합니다.

- 동일 퍼즐 집합 최소 5개를 확정하고, 알고리즘별로 같은 순서로 실행합니다.
- 실행 시간은 초(second) 단위로 기록하고, 가능한 소수점 3자리까지 표기합니다.
- 각 알고리즘별 평균 시간(Mean)과 최대 시간(Max)을 계산합니다.
- 시간 비교 표 1개 + 막대그래프 1개를 작성합니다.

감점 방지: 서로 다른 퍼즐로 알고리즘을 비교하면 공정 비교로 인정되지 않습니다.

Task 4. 경로 길이 비교

목표: 어떤 알고리즘이 더 짧은 해를 찾는지 분석합니다.

- Task 3와 동일 퍼즐을 사용해 알고리즘별 경로 길이(액션 개수)를 기록합니다.
- 퍼즐별 최소 경로 길이를 찾고, 각 알고리즘이 그 값과 얼마나 차이 나는지 비교합니다.
- "빠르지만 경로가 길다/느리지만 경로가 짧다" 같은 trade-off를 서술합니다.

보고서 필수 포함: 퍼즐별 경로 길이 표 + 핵심 해석 3문장 이상.

Task 5. A*와 휴리스틱 분석

목표: 왜 A*가 빠른 수 있는지 휴리스틱 관점에서 설명합니다.

- heuristic()가 무엇을 계산하는지 수식/설명으로 명시합니다.
- 예시 상태 2개 이상에서 heuristic 값 계산 결과를 직접 보여줍니다.
- A*의 우선순위가 $g(n)+h(n)$ 으로 결정됨을 설명하고, 실행 결과와 연결합니다.
- 휴리스틱이 부정확하면 어떤 문제가 생길 수 있는지 짧게 정리합니다.

보고서 필수 포함: 휴리스틱 계산 예시, A* 성능 해석, 한계 1가지 이상.

Task 6. DFS 한계 분석

목표: DFS가 왜 비교 기준으로 부적합할 수 있는지 근거로 설명합니다.

- DFS가 비효율적이었던 퍼즐 사례를 최소 1개 선택합니다.
- 동일 퍼즐에서 BFS/UCS/A* 결과와 나란히 비교합니다.
- DFS가 긴 경로를 만들거나 오래 걸린 원인을 탐색 순서 관점에서 설명합니다.

보고서 필수 포함: DFS 문제 사례(수치 포함) + 원인 분석 문장.

Task 7. 공정성 실험 설계

목표: 재현 가능한 실험 프로토콜을 제시합니다.

- 동일 입력 퍼즐, 동일 실행 환경, 동일 종료 조건을 명시합니다.
- 측정 항목(시간, 경로 길이, 성공 여부)과 측정 방법을 먼저 고정합니다.
- 실험 절차를 순서도로 쓰거나 1~2~3 단계로 기술합니다.
- 재실행했을 때 결과가 얼마나 유사한지 간단히 점검합니다.

보고서 필수 포함: 실험 조건 표, 절차 설명, 재현성 관련 코멘트.

Task 8. 난이도-성능 관계

목표: 문제 난이도가 성능에 미치는 영향을 설명합니다.

- 난이도 기준을 직접 정의합니다(예: 쉬이 ힳ수, 목표까지 추정 거리).
- 쉬움/중간/어려움 그룹으로 퍼즐을 분류합니다.
- 그룹별 평균 시간과 평균 경로 길이를 계산합니다.
- 난이도 증가에 따라 어떤 알고리즘이 급격히 느려지는지 분석합니다.

보고서 필수 포함: 난이도 정의, 그룹별 통계표, 핵심 인사이트.

Task 9. 4x4, 5x5 확장 분석

목표: 8-puzzle 코드를 일반 N-puzzle로 확장할 때의 핵심 변경점을 정리합니다.

- 상태 표현(리스트 크기, 인덱스 계산, 목표 상태 생성) 일반화 방향을 작성합니다.
- search.py 내부에서 가능한 action 처리와 successor 탐색 로직을 어떻게 일반화할지 정리합니다.
- goal 상태 판정이 일반화될 때 탐색 로직이 어떤 영향을 받는지 설명합니다.
- 휴리스틱(Manhattan distance) 계산을 N 크기에 맞게 일반화하는 방법을 설명합니다.
- 메모리/시간 복잡도가 왜 급격히 증가하는지 원인과 대응 아이디어를 제시합니다.

보고서 필수 포함: 코드 변경이 아닌 설계안(개념), 예상 병목, 개선 방향 2가지 이상.

9. 평가 기준

평가 항목	설명
알고리즘 이해	알고리즘 원리와 차이를 정확히 설명했는가
구현 정확성	코드가 기본 동작을 수행하는가
실험 설계	비교가 공정하고 재현 가능한가
근거 자료	표, 수치, 사례가 충분한가
분석 깊이	결과를 해석하고 이유를 설명했는가
문서 완성도	보고서가 읽기 쉽게 정리되었는가
AI 공개	AI 활용 내용을 구체적으로 밝혔다는가

10. 제출 전 확인 사항

☐ search.py를 제출했는가

☐ search.py 외 파일을 수정하지 않았는가

☐ 보고서 PDF를 제출했는가

☐ Task 1 ~ Task 9를 모두 수행했는가

☐ 결과 표 또는 그래프가 포함되었는가

☐ 분석과 결론이 포함되었는가

☐ AI 사용 공개를 작성했는가